CAHIER DES CHARGES

Comprendre et Anticiper les Prix des Fruits & Légumes

Certification RNCP37827 — Développeur en Intelligence Artificielle
Simplon — Promotion 2026

Leila BOUBEKEUR

Sommaire

Sommaire 2
1. Contexte & Enjeux du projet 4
1.1 Contexte général 4
1.2 Hypothèse centrale 4
1.3 Objectifs du projet 4
2. Choix du projet et justification 5
2.1 Choix du sujet IA 5
2.2 Périmètre fonctionnel 5
3. Spécifications fonctionnelles 6
3.1 Variable cible et features 6
3.2 Métriques de succès 6
4. User Stories & Backlog 7
4.1 Epics 7
4.2 User Stories détaillées 7
Epic 1 — Collecte & ETL 7
Epic 2 — Machine Learning 8
Epic 3 — API REST 8
Epic 4 — Dashboard Streamlit 8
Epic 5 — MLOps & DevOps 9
5. Architecture technique 10
5.1 Choix d'architecture — Architecture N-tiers 10
5.2 Modélisation des données — Merise (MCD) 12
5.3 Modélisation des données — Merise (MPD) 13
Le MLD traduit le MPD en tables relationnelles PostgreSQL. Chaque entité devient une table avec ses clés primaires et étrangères. 13
5.3 Pipeline de données 14
6. Planification agile 15
6.1 Méthode Scrum/Kanban 15
6.2 Planning prévisionnel 15
6.3 Suivi et organisation du travail 15
7. Maquettes du dashboard 16
7.1 Page Accueil 16
7.2 Page Prix & Tendances 17
7.3 Page Prédiction 18
8. Mapping des compétences RNCP 37827 19
Annexe — Benchmark des outils 20
A. Langages & traitement des données 20
B. Modèles Machine Learning 20
C. Base de données 20
D. API REST 21

E. Dashboard 21
F. Automatisation & Veille 21
G. CI/CD & Conteneurisation 22
H. Monitoring ML 22
I. Gestion de projet 22
Annexe — Sources de données 23

1. Contexte & Enjeux du projet

1.1 Contexte général

Les prix des fruits et légumes sont soumis à de fortes variations saisonnières et conjoncturelles, rendant difficile la planification pour les consommateurs, les distributeurs et les producteurs. L'inflation alimentaire des années 2021-2023, accentuée par les crises énergétiques et les perturbations des chaînes d'approvisionnement, a mis en évidence le besoin d'outils prédictifs fiables.

Ce projet s'inscrit dans une démarche d'intelligence artificielle appliquée à l'agroalimentaire, visant à croiser des données hétérogènes — prix de détail, météo, énergie, engrais, import/export — pour construire un modèle prédictif performant.

1.2 Hypothèse centrale

Les prix des fruits et légumes ne sont pas aléatoires. Ils résultent mécaniquement de quatre variables principales :

- Les conditions météorologiques dans les zones de production
- Les coûts énergétiques de transport et de stockage
- Les prix des intrants agricoles (engrais) avec un décalage temporel de 3 à 6 mois
- Les volumes d'import/export selon les pays partenaires

Un modèle ML entraîné sur ces variables croisées peut capturer les signaux prédictifs avec un décalage temporel exploitable.

1.3 Objectifs du projet

- Concevoir un pipeline ETL complet de collecte et d'enrichissement de données multi-sources
- Entraîner plusieurs modèles de Machine Learning pour sélectionner le plus performant
- Exposer le modèle via une API REST FastAPI documentée
- Développer un dashboard interactif Streamlit pour la visualisation et la prédiction
- Mettre en place un système de veille automatisée via N8N et Notion
- Implémenter un pipeline CI/CD avec GitHub Actions

2. Choix du projet et justification

2.1 Choix du sujet IA

Le projet porte sur la prédiction des prix des fruits et légumes aux États-Unis à partir des données USDA ERS (Economic Research Service). Ce choix est motivé par :

- La disponibilité d'un dataset public riche et fiable (USDA ERS, 710 lignes, 24 colonnes)
- La pertinence sociale du sujet — impact direct sur le pouvoir d'achat alimentaire
- La complexité technique adaptée à la certification DevIA — données tabulaires, enrichissement multi-sources, ML supervisé
- L'opportunité de valider l'ensemble des 21 compétences RNCP 37827 sur un projet cohérent

2.2 Périmètre fonctionnel

Fonctionnalité	Description
Collecte données	ETL complet — USDA ERS, Open-Meteo, EIA, World Bank, NASS
Enrichissement	Ajout variables météo, énergie, engrais, import/export
Modèle ML	Gradient Boosting — R^2=0.92 — prédiction prix cup équivalent
API REST	FastAPI POST /predict — documentation OpenAPI automatique
Dashboard	Streamlit — 3 pages — visualisation, prédiction
Veille automatisée	N8N — 5 flux RSS — export quotidien Notion à 8h
CI/CD	GitHub Actions — lint, test, build Docker, déploiement
Monitoring ML	MLflow — suivi métriques, détection drift, alertes

3. Spécifications fonctionnelles

3.1 Variable cible et features

Catégorie	Variables	Source
Variable cible (y)	prix_cup — Prix par cup équivalent	USDA ERS
Météo	temp_grow_avg, frost_days, precip_total, heat_days	Open-Meteo
Énergie	diesel_price, electricity_price, gas_price	EIA
Engrais	urea_price, dap_price, tsp_price (lag 3-6 mois)	World Bank
Commerce	import_volume, export_volume, origin_country	USDA FAS
Production	production_lbs, yield_per_acre, harvested_acres	USDA NASS
Temporel	month, season, year, is_growing_season	Calculé
Produit	product_type, form (fresh/frozen/canned), category	USDA ERS

3.2 Métriques de succès

Critère	Objectif
Prédiction	RMSE < 0.30 $/cup
Précision	$R^2 > 0.85$
Interprétabilité	Top 5 SHAP cohérentes
API	Latence < 200ms
MLOps	Pipeline CI/CD fonctionnel

4. User Stories & Backlog

4.1 Epics

Le backlog est organisé en 5 Epics couvrant l'ensemble des fonctionnalités du projet :

- Epic 1 — Collecte & ETL des données (C1, C3)
- Epic 2 — Modélisation & Machine Learning (C7, C8)
- Epic 3 — API REST & Mise à disposition (C5, C9)
- Epic 4 — Dashboard & Interface utilisateur (C10, C17)
- Epic 5 — MLOps & DevOps (C11, C13, C18, C19)

User Stories

Table | Nouveau

Nom	ID	Epic	Priorité	Statut	Sprint	Critères d'acceptation
EPIC DONNEES						
Compiler les données ERS de 5 années en un seul fichier	US - 01	Données	MUST	Terminé	Sprint 1	ers_toutes_annees.csv généré — 704 lignes — colonnes produit, forme, prix_detail, rendement, prix_cup, categorie, annee
Enrichir les données via l'API NASS pour connaître l'état producteur	US - 02	Données	MUST	Terminé	Sprint 1	Colonne etat_production ajoutée — taux de couverture >= 25% — données disponibles par année
Récupérer les données météo historiques par état producteur	US - 03	Données	MUST	Terminé	Sprint 1	Colonnes temp_moyenne, jours_gel, jours_chaleur, precip_totale — par état et par année
Récupérer les prix du diesel et de l'électricité par année	US - 04	Données	MUST	Terminé	Sprint 2	Colonnes prix_diesel, prix_electricite — 5 années couvertes — source API EIA

Capture d'écran : Backlog Notion avec toutes les user stories organisées par Epic et statut

4.2 User Stories détaillées

Epic 1 — Collecte & ETL

ID	User Story	Critère d'acceptation	Priorité
US-01	En tant qu'analyste, je veux importer le dataset USDA ERS pour disposer des données de base	710 lignes chargées, 0 erreur	Must
US-02	En tant que développeur, je veux nettoyer les données pour garantir la qualité du dataset	0 doublon, valeurs manquantes < 5%	Must
US-03	En tant qu'analyste, je veux enrichir les données météo pour améliorer les prédictions	Variables météo présentes pour chaque ligne	Must
US-04	En tant qu'analyste, je veux ajouter les prix d'énergie pour capturer les coûts de transport	diesel_price, electricity_price présents	Should

US-05	En tant qu'analyste, je veux ajouter les prix d'engrais avec lag temporel	urea_price_lag_6m calculé	Should

Epic 2 — Machine Learning

ID	User Story	Critère d'acceptation	Priorité
US-06	En tant que data scientist, je veux benchmarker 5 modèles ML pour choisir le meilleur	Tableau comparatif RMSE/R²/MAE	Must
US-07	En tant que data scientist, je veux exporter le modèle en .pkl pour le déploiement	Fichier model.pkl généré et chargeable	Must
US-08	En tant que data scientist, je veux générer les graphiques SHAP pour expliquer les prédictions	Top 5 features SHAP affichées	Must
US-09	En tant qu'utilisateur, je veux comprendre pourquoi le prix est prédit ainsi	Explication SHAP visible dans le dashboard	Should

Epic 3 — API REST

ID	User Story	Critère d'acceptation	Priorité
US-10	En tant que développeur, je veux un endpoint POST /predict pour obtenir une prédiction	Réponse JSON < 200ms	Must
US-11	En tant que développeur, je veux une documentation OpenAPI générée automatiquement	Swagger UI accessible sur /docs	Must
US-12	En tant qu'utilisateur, je veux que l'API gère les erreurs proprement	Codes HTTP 400/422/500 documentés	Should

Epic 4 — Dashboard Streamlit

ID	User Story	Critère d'acceptation	Priorité
US-13	En tant qu'utilisateur, je veux visualiser les prix par catégorie de produits	Graphiques interactifs affichés	Must
US-14	En tant qu'utilisateur, je veux prédire le prix d'un produit via un formulaire	Formulaire fonctionnel, prix affiché	Must
US-15	En tant qu'utilisateur, je veux consulter les articles de veille depuis le dashboard	Page veille affichant les articles Notion	Should
US-16	En tant qu'utilisateur, je veux que le dashboard soit accessible sur mobile	Design responsive, WCAG 2.1 AA	Could

Epic 5 — MLOps & DevOps

ID	User Story	Critère d'acceptation	Priorité
US-17	En tant que DevOps, je veux un pipeline CI/CD automatisé sur chaque push Git	GitHub Actions lint+test+build OK	Must
US-18	En tant que data scientist, je veux tracker les métriques du modèle avec MLflow	Dashboard MLflow actif, métriques loggées	Must
US-19	En tant que DevOps, je veux que les services soient conteneurisés avec Docker	docker-compose up lance tous les services	Must

5. Architecture technique

5.1 Choix d'architecture — Architecture N-tiers

Le projet adopte une architecture N-tiers (multicouche) qui sépare clairement les responsabilités en couches indépendantes. Ce choix est justifié par :

- La séparation claire entre interface utilisateur, logique applicative, persistance des données et services IA
- La maintenabilité et l'évolutivité de chaque couche indépendamment
- La facilité de déploiement via Docker et GitHub Actions
- La compatibilité avec les exigences MLOps de la certification

Couche	Composant	Rôle
Présentation	Streamlit	Dashboard interactif — visualisation et prédiction
Application	FastAPI	API REST — exposition du modèle ML
Métier / IA	scikit-learn + SHAP	Modèle Gradient Boosting — prédiction et interprétabilité
Persistance	PostgreSQL	Stockage relationnel — données enrichies
Infrastructure	Docker	Conteneurisation — reproductibilité des environnements
CI/CD	GitHub Actions	Tests automatisés — build — déploiement
Monitoring ML	MLflow	Suivi métriques — détection drift — alertes
Veille	N8N + Notion	Automatisation — flux RSS — export quotidien

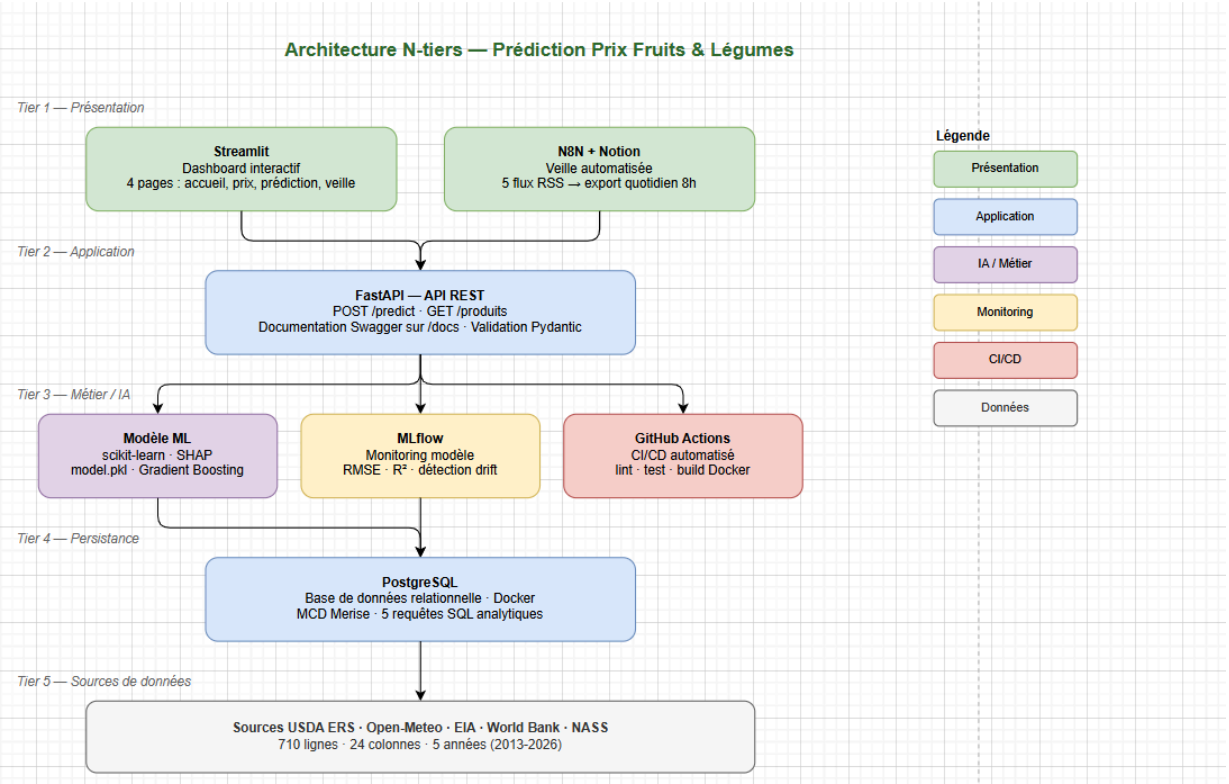
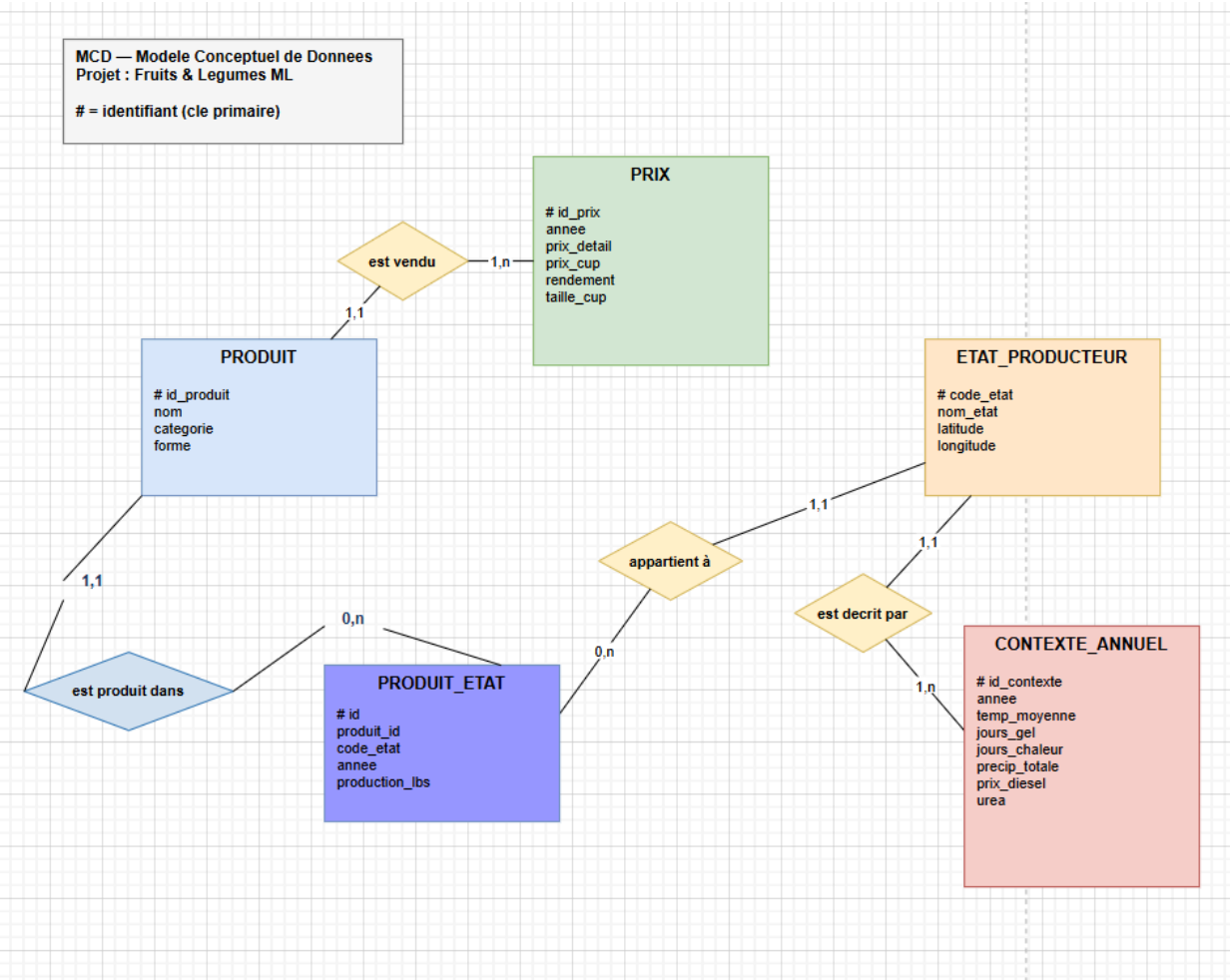


Diagramme d'architecture N-tiers — draw.io ou capture de l'architecture globale

5.2 Modélisation des données — Merise (MCD)

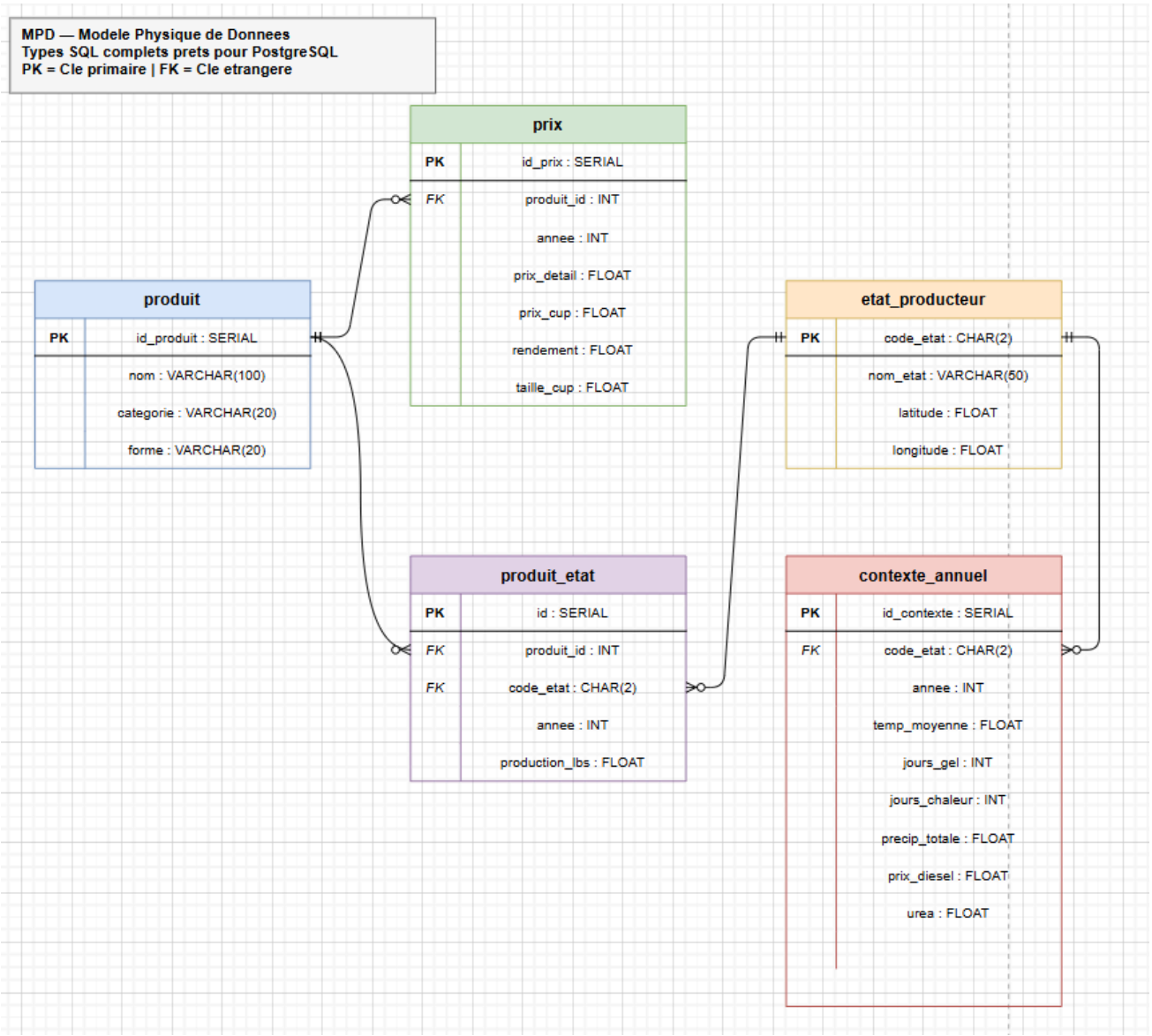
La modélisation des données suit la méthode Merise avec un Modèle Conceptuel de Données (MCD) comportant 5 entités et 4 associations.



MCD draw.io — 5 entités : PRODUIT, PRIX, PRODUIT_ETAT, ETAT_PRODUCTEUR, CONTEXTE_ANNUEL

5.3 Modélisation des données — Merise (MPD)

Le MLD traduit le MPD en tables relationnelles PostgreSQL. Chaque entité devient une table avec ses clés primaires et étrangères.



MPD draw.io

5.3 Pipeline de données

Le pipeline complet suit 10 étapes de la collecte au déploiement :

Étape	Action	Notebook / Outil
1	Chargement dataset USDA ERS	01_ETL_FruitsLegumes.ipynb
2	Nettoyage et normalisation	01_ETL_FruitsLegumes.ipynb
3	Enrichissement météo (Open-Meteo)	02_Enrichissement.ipynb
4	Enrichissement énergie (EIA)	02_Enrichissement.ipynb
5	Enrichissement engrais (World Bank)	02_Enrichissement.ipynb
6	Analyse exploratoire (EDA)	03_Visualisation_EDA.ipynb
7	Benchmark 5 modèles ML	04_Modele_ML.ipynb
8	Export modèle .pkl + SHAP	04_Modele_ML.ipynb
9	API REST FastAPI /predict	api/main.py
10	Dashboard Streamlit	app/streamlit_app.py

6. Planification agile

6.1 Méthode Scrum/Kanban

Le projet est piloté selon une méthode agile Scrum/Kanban avec des sprints de 2 semaines. Le backlog est géré dans Notion avec les colonnes : À faire, En cours, En revue, Terminé.

> *Capture d'écran à insérer ici : Tableau Kanban Notion avec les sprints et les user stories*

6.2 Planning prévisionnel

Phase	Période	Contenu
Phase 1	Semaines 1-2	ETL, nettoyage, EDA complète — C1, C3
Phase 2	Semaines 3-4	Enrichissement multi-sources, PostgreSQL — C2, C4
Phase 3	Semaines 5-7	ML, SHAP, export .pkl, veille N8N — C6, C7, C8
Phase 4	Semaines 8-9	FastAPI /predict, tests — C5, C9
Phase 5	Semaines 10-11	Streamlit dashboard — C10, C17
Phase 6	Semaines 12-13	GitHub Actions CI/CD, MLflow — C11, C18, C19
Phase 7	Semaines 14-16	Rapports, soutenance, préparation oral

> *Capture d'écran : Planning Notion avec les phases du projet*

6.3 Suivi et organisation du travail

Le projet étant réalisé de manière individuelle, la méthode agile est adaptée au contexte mono-développeur. Le suivi repose sur :

- **Revue hebdomadaire** (1h) : bilan des tâches réalisées dans la semaine, mise à jour du backlog Notion, identification des blocages
- **Validation formateur** : présentation de l'avancement à chaque fin de sprint (toutes les 2 semaines) avec démonstration des fonctionnalités livrées
- **Backlog Notion** : mise à jour quotidienne des statuts (À faire / En cours / Terminé)
- **Journal de bord** : note des décisions techniques et des obstacles rencontrés à chaque session de travail
- **Git comme traçabilité** : chaque commit correspond à une fonctionnalité ou correction identifiable

7. Maquettes du dashboard

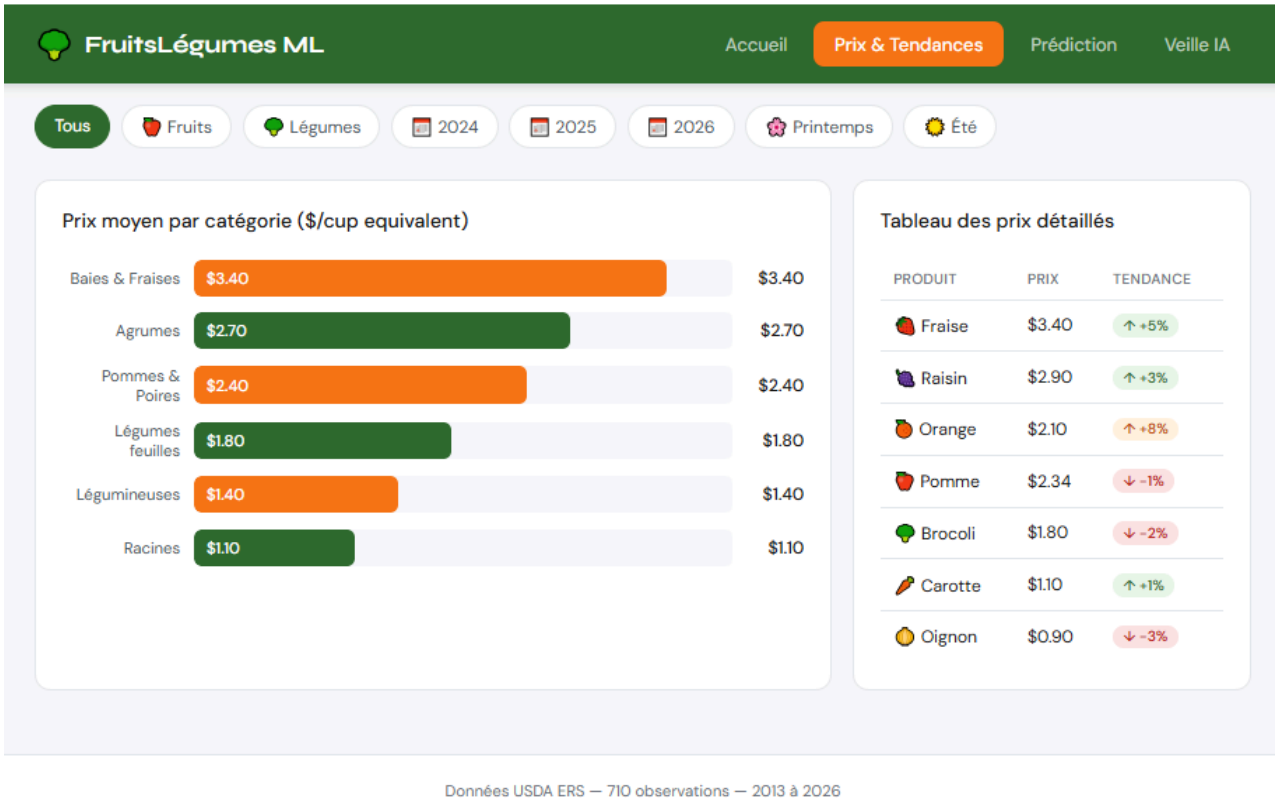
7.1 Page Accueil

La page d'accueil présente le projet, les métriques clés et les produits phares avec leurs prix actuels.

Maquette Figma : Page Accueil — bandeau vert/orange, emojis fruits & légumes, 4 métriques, top produits

7.2 Page Prix & Tendances

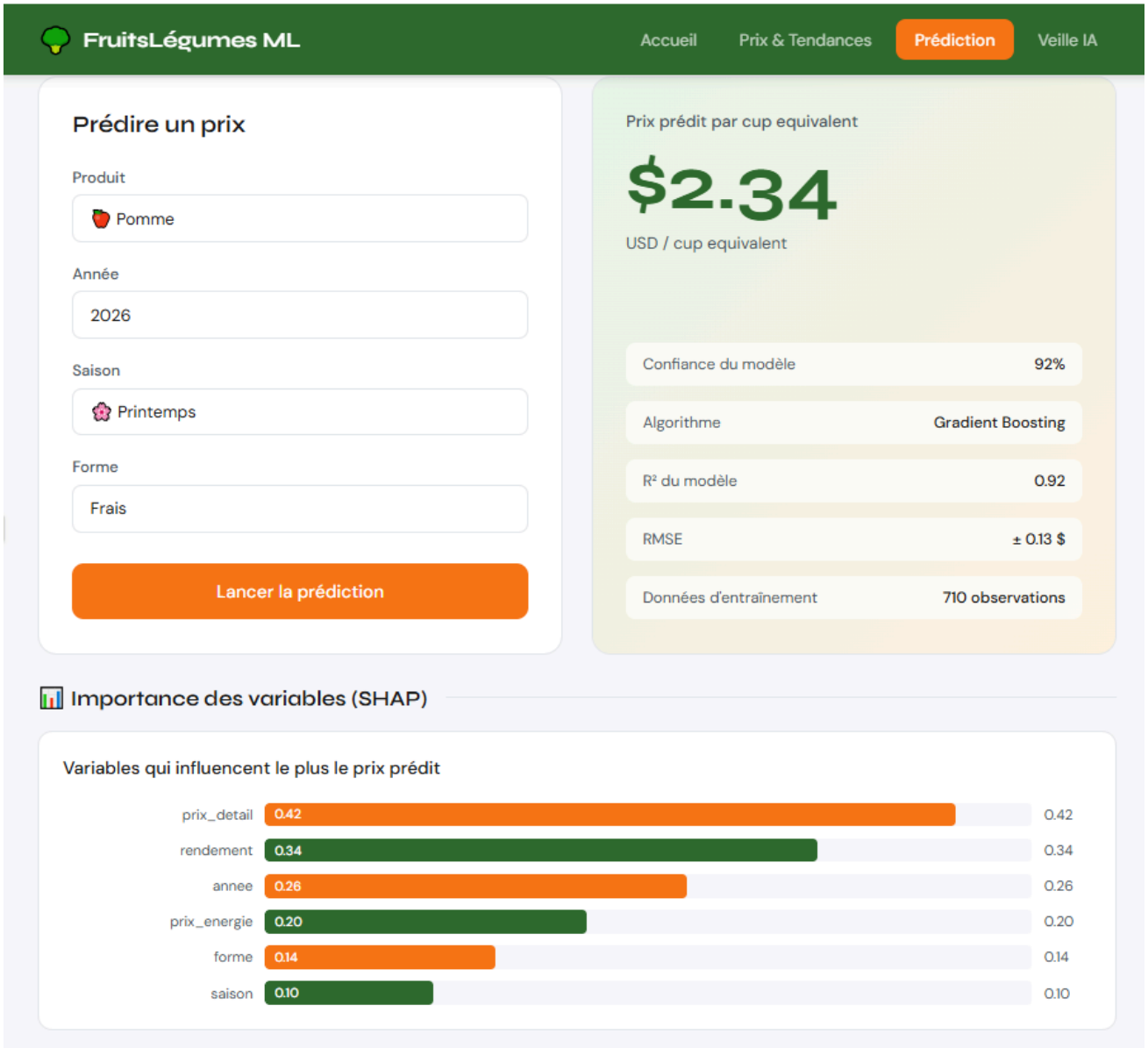
Visualisation des prix par catégorie avec graphiques en barres, filtres par saison et tableau des prix détaillés.



Maquette Figma : Page Prix & Tendances — graphiques, filtres, tableau

7.3 Page Prédiction

Formulaire de saisie permettant de prédire le prix d'un produit selon sa catégorie, l'année, la saison et la forme (frais/congelé/conserve). Résultat accompagné du graphique SHAP.



Maquette Figma : Page Prédiction — formulaire, résultat prix, graphique SHAP

8. Mapping des compétences RNCP 37827

Comp.	Intitulé	Application dans ce projet	Épreuve
C1	Extraction multi-sources	ETL — 6 sources : USDA, météo, énergie, engrais, NASS, FAS	E1
C2	Requêtes SQL	5 requêtes analytiques sur PostgreSQL	E1
C3	Agrégation données	Nettoyage, encodage, imputation, jointures	E1
C4	Base de données Merise	MCD + MLD + script SQL + import PostgreSQL	E1
C5	API REST FastAPI	POST /predict — OpenAPI — tests Pytest	E1+E2
C6	Veille technique	N8N — 5 flux RSS — enrichissement auto — Notion	E2
C7	Benchmark services IA	5 modèles benchmarkés — SHAP features	E2
C8	POC service IA	Entraînement du modèle retenu — validation en pré-production — export .pkl	E2
C9	API modèle	FastAPI POST /predict — auth JWT	E2+E3
C10	Intégration API	Streamlit consomme FastAPI /predict	E3
C11	Monitoring modèle	MLflow — RMSE prod, drift detection	E3
C16	Agilité	Backlog Notion — Sprints — Kanban	E4
C17	Dev application	Streamlit — 3 pages	E4
C18	CI automatisée	GitHub Actions — lint + test + build	E4
C19	Livraison continue	Pipeline CD — Docker — déploiement staging	E4

Annexe — Benchmark des outils

A. Langages & traitement des données

Outil	Avantages	Inconvénients	Choix retenu
Python	Écosystème ML riche, pandas, scikit-learn	Performance inférieure à Java/Go	Oui — standard industrie ML
R	Statistiques avancées	Moins adapté au déploiement	Non
Julia	Très rapide	Écosystème ML limité	Non

B. Modèles Machine Learning

Modèle	Explication du modèle	Métriques évaluées
Gradient Boosting	Très performant sur données tabulaires, gère bien les relations non linéaires	RMSE, MAE, R^2
Random Forest	Référence robuste, bon point de comparaison	RMSE, MAE, R^2
XGBoost	Version optimisée du boosting, souvent plus rapide	RMSE, MAE, R^2
Régression linéaire	Modèle de base simple — sert de référence minimale	RMSE, MAE, R^2
Ridge	Régression linéaire avec régularisation — meilleure généralisation	RMSE, MAE, R^2

C. Base de données

Outil	Avantages	Inconvénients	Choix retenu
PostgreSQL	Open-source, robuste, SQL complet, Docker facile	Configuration initiale requise	Oui — standard données relationnelles
MySQL	Très répandu, rapide	Moins de fonctionnalités avancées	Non
SQLite	Zéro configuration	Pas adapté à la production	Non — pas scalable
MongoDB	Flexible, NoSQL	Pas adapté aux données tabulaires structurées	Non

D. API REST

Outil	Avantages	Inconvénients	Choix retenu
FastAPI	Python natif, OpenAPI auto, très rapide, validation Pydantic	Moins mature que Flask	Oui — standard ML APIs
Flask	Simple, bien documenté	Pas de validation auto, plus lent	Non
Django REST	Framework complet	Trop lourd pour une simple API ML	Non

E. Dashboard

Outil	Avantages	Inconvénients	Choix retenu
Streamlit	Python pur, rapide à développer, intégration ML facile	Moins flexible que React	Oui — idéal pour ML dashboard
Gradio	Très rapide pour démos	Peu personnalisable	Non
React	Très flexible, production-grade	Nécessite JavaScript, plus complexe	Non — hors périmètre
Dash (Plotly)	Graphiques interactifs avancés	Plus verbeux que Streamlit	Non

F. Automatisation & Veille

Outil	Avantages	Inconvénients	Choix retenu
N8N	Open-source, visuel, self-hébergé Docker, nombreux connecteurs	Nécessite Docker, apprentissage initial	Oui — veille automatisée RSS → Notion
Zapier	Très simple, sans code	Payant, données dans le cloud	Non — payant
Make (Integromat)	Puissant, visuel	Payant au-delà de 1000 opérations/mois	Non — payant
Script Python	Contrôle total, gratuit	Exécution manuelle, maintenance complexe	Complémentaire

G. CI/CD & Conteneurisation

Outil	Avantages	Inconvénients	Choix retenu
Docker	Reproductibilité totale, standard industrie	Courbe d'apprentissage initiale	Oui — conteneurisation complète
GitHub Actions	Intégré à GitHub, gratuit, YAML simple	Limité à 2000 min/mois en gratuit	Oui — CI/CD automatisé
Jenkins	Très flexible, self-hébergé	Configuration complexe	Non
GitLab CI	Intégré, puissant	Nécessite GitLab	Non — projet sur GitHub

H. Monitoring ML

Outil	Avantages	Inconvénients	Choix retenu
MLflow	Open-source, suivi expériences, model registry	Interface basique	Oui — standard MLOps
Weights & Biases	Interface riche, collaboration	Payant au-delà du tier gratuit	Non
DVC	Versioning données et modèles	Complémentaire, pas de dashboard	Complémentaire

I. Gestion de projet

Outil	Avantages	Inconvénients	Choix retenu
Notion	Backlog, veille, documentation centralisée, gratuit	Moins spécialisé que Jira	Oui — backlog + veille
GitHub Projects	Intégré au repo, Kanban natif	Interface limitée	Complémentaire
Jira	Standard entreprise, puissant	Complexe, payant en équipe	Non
Trello	Simple, visuel	Peu de fonctionnalités avancées	Non

Annexe — Sources de données

Source	Données fournies	Justification
USDA ERS	Prix détail 150+ produits, 2013-2023	Source officielle américaine — variable cible
Open-Meteo	Météo historique par zone de production	Gratuit, API REST, conditions climatiques production
EIA	Prix diesel, électricité, gaz par État	Coûts transport & stockage — impact prix
World Bank	Pink Sheet — prix engrais mondiaux	Lag 3-6 mois — feature cruciale pour prédiction
USDA NASS	Production, rendement, surfaces par État	Volumes offre — corrélation prix
USDA FAS	Import/Export par produit et pays	Produits importés vs domestiques